> Backtick strings that support embedded expressions, multi-line text, and cleaner string composition — a massive upgrade over traditional string concatenation.

The Problem Template Literals Solve

Before template literals, building dynamic strings required painful concatenation with `+`:

```
const name = "mona";
const age = 30;

// Old way — hard to read, easy to make mistakes
console.log('My name is ' + name + ' and I am ' + age + ' years
old');
// "My name is mona and I am 30 years old"

// Template literal — clean and readable
console.log(`My name is ${name} and I am ${age} years old`);
// "My name is mona and I am 30 years old"
```

> **Syntax:** Use backticks `` ` `` instead of single `'` or double `"` quotes.

Feature 1 — String Interpolation `${}`

Embed any JavaScript expression directly inside a string using `${}`:

```
const name = "mona";
const age = 30;

// Variable
console.log(`Hello, ${name}!`); // "Hello, mona!"

// Expression
console.log(`In 5 years, she will be ${age + 5}`); // "In 5 years,
```

```
she will be 35"

// Function call
console.log(`Name uppercase: ${name.toUpperCase()}`); // "Name
uppercase: MONA"

// Ternary operator
const isAdult = age >= 18;
console.log(`Status: ${isAdult ? "adult" : "minor"}`); // "Status:
adult"

// Math
console.log(`Pi approx: ${(22/7).toFixed(4)}`); // "Pi approx:
3.1429"
```

> **Anything inside `${}` is evaluated as JavaScript** — variables, math, function calls, ternaries, even other template literals. The result is converted to a string and inserted.

Feature 2 — Multi-line Strings

Old strings required `\n` escape characters and `+` concatenation for multi-line text:

```
// Old way — ugly
const old = 'line 1 \n' +
            'line 2 \n' +
            'line 3 \n';
console.log(old);
// line 1
// line 2
// line 3

// Template literal — just press Enter
const multiLine = `
line 1
line 2
line 3
`;
```

```
console.log(multiLine);
// (blank line)
// line 1
// line 2
// line 3
// (blank line)
```

> **Note:** The newline immediately after the opening backtick is included in the string. If you don't want a leading blank line, start your content on the same line as the backtick, or use `.trim()`.

```
// Starting content on the same line avoids the leading newline
const clean = `line 1
line 2
line 3`;
```

Feature 3 — No More Quote Escaping

Old strings broke when the string content contained the same quote character:

```
// Old way - had to escape or switch quote types
console.log('my name \'\'');       // escaping
console.log("it's a great day");   // switching quote types

// Template literal - backticks don't conflict with ' or "
console.log(`my name ''`);         // no escaping needed
console.log(`it's a "great" day`); // single and double quotes freely
used
```

Common Mistake — Nesting Backticks in `${}`

You **can** nest template literals inside `${}`, but the syntax must be correct:

```
const age = 30;
```

```
// WRONG — the backtick inside ${} closes the outer template literal
console.log(` my age ${age `years`} `); // SyntaxError

// CORRECT — use a regular string inside ${}
console.log(` my age ${age + " years"} `); // "my age 30 years"

// CORRECT — nest a template literal properly
console.log(`my age ${`${age} years`}`); // "my age 30 years"
```

Practical Use Cases

Building HTML Strings

Template literals shine when building HTML dynamically:

```
const user = { name: "Omar", age: 30, email: "omar@gmail.com" };

const card = `
  <div class="user-card">
    <h2>${user.name}</h2>
    <p>Age: ${user.age}</p>
    <p>Email: ${user.email}</p>
  </div>
`;

document.body.innerHTML = card;
```

Building URLs

```
const baseUrl = "https://api.example.com";
const userId = 42;
const endpoint = `${baseUrl}/users/${userId}/profile`;
// "https://api.example.com/users/42/profile"
```

Conditional Content

```
const score = 85;
const message = `You scored ${score}/100. ${score >= 60 ? "You
passed!" : "You failed."}`;
// "You scored 85/100. You passed!"
```

Multi-line SQL / Queries

```
const query = `
  SELECT name, email
  FROM users
  WHERE age > ${minAge}
  ORDER BY name ASC
`;
```

Logging with Context

```
function divide(a, b) {
  if (b === 0) throw new Error(`Cannot divide ${a} by zero`);
  return a / b;
}
```

Template Literals vs String Concatenation

Feature	Concatenation `+`	Template Literal
Readability	Hard with many variables	Very readable
Multi-line	Requires `\n` and `+`	Just press Enter
Expressions	Only variables work cleanly	Any JS expression
Quote conflicts	Need escaping	No conflicts
Performance	Same	Same

Advanced — Tagged Template Literals

A more advanced feature: you can prefix a template literal with a function name (a "tag") to process it:

```
function highlight(strings, ...values) {
  return strings.reduce((result, str, i) => {
    return result + str + (values[i] ? `<strong>${values[i]}
</strong>` : '');
  }, '');
}

const name = "mona";
const age = 30;

const output = highlight`My name is ${name} and I am ${age} years
old`;
// "My name is <strong>mona</strong> and I am <strong>30</strong>
years old"
```

> Tagged templates are used in real-world libraries like `styled-components` (CSS-in-JS), `graphql` (query parsing), and `sql` (safe query building).

Quick Reference

```
// Basic interpolation
`Hello, ${name}!`

// Expression
`Result: ${a + b}`

// Function call
`Upper: ${str.toUpperCase()}`

// Ternary
`${x > 0 ? "positive" : "negative"}`

// Multi-line
`line 1
 line 2
```

```
 line 3`

// Single and double quotes — no escaping
`it's a "beautiful" day`

// Nested template literal
`outer ${`inner ${value}`} outer`
```

Summary

```
Template Literals (backticks ``) give you:
  1. Interpolation    →  ${expression}  — embed any JS expression
  2. Multi-line       →  just press Enter, no \n needed
  3. No quote issues  →  use ' and " freely inside backticks
  4. Tagged templates →  advanced processing with a tag function
```

Abdullah Ali

Contact : +201012613453